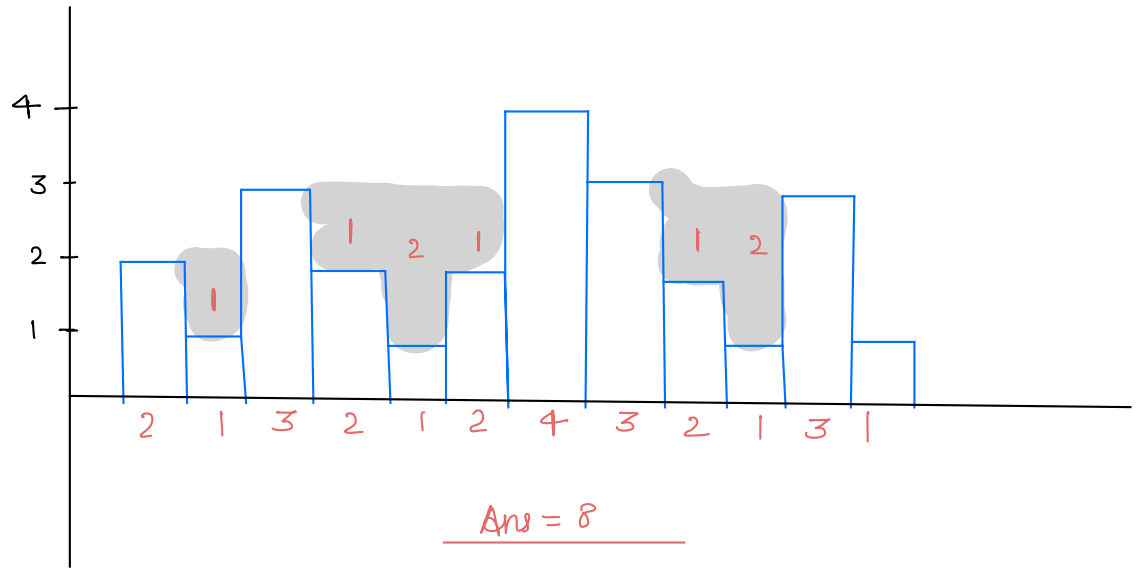
Qu Given n buildings with height of each building find rain water trapped b/w buildings.

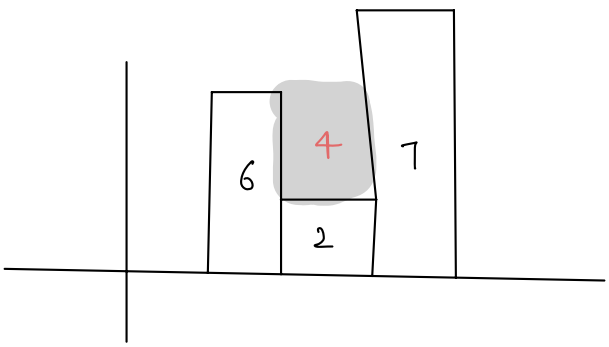
Microsoft
Amazon
Apple
Oracle

2	1	3	2	1	2	4	3	2	1	3	1

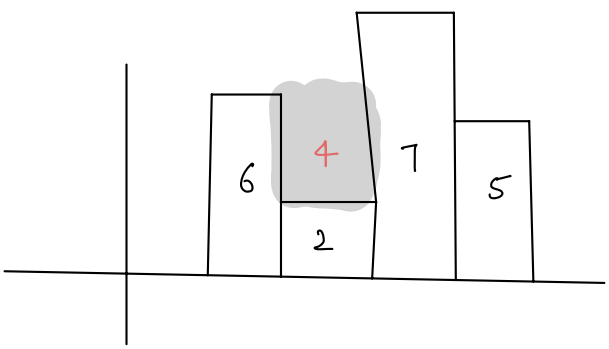


Ans = 8

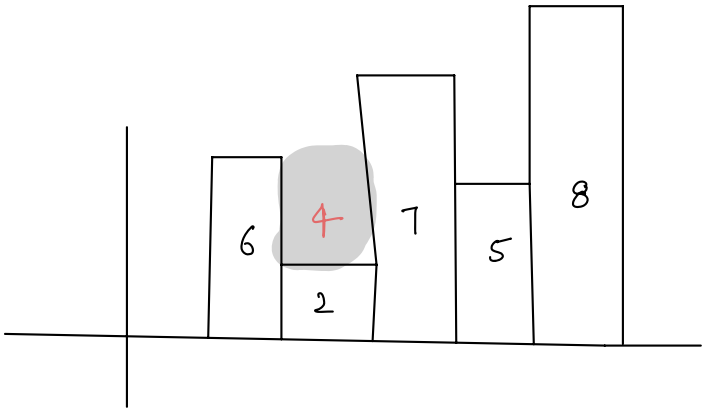
Observations



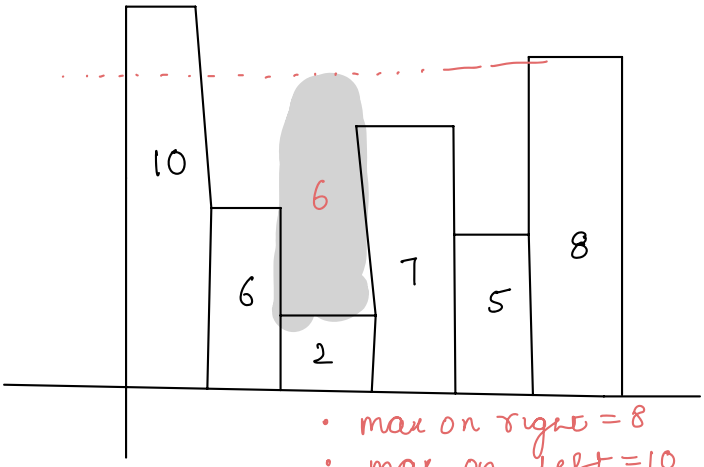
Water stored over building 2 ?



Water stored over building 2 ?



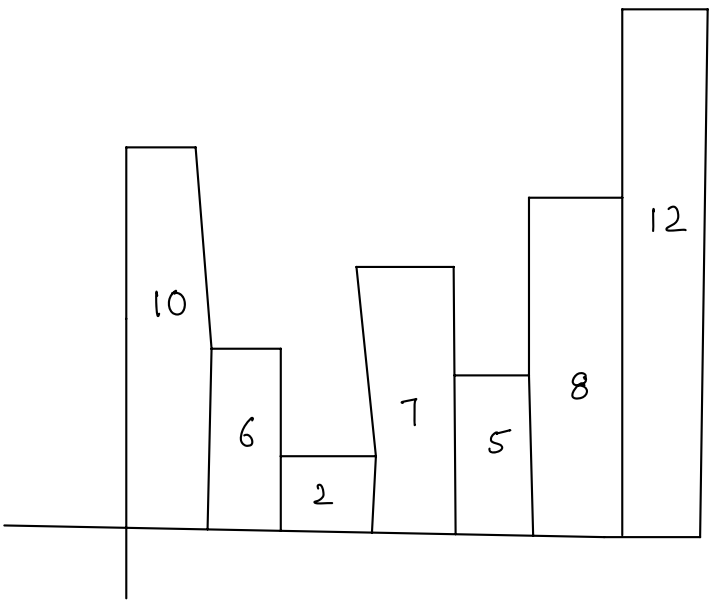
Water stored over building 2 ?



Water stored over building 2 ?

- max on right = 8
- max on left = 10

min (8, 10) − 2 = 8 − 2 = 6 Ans



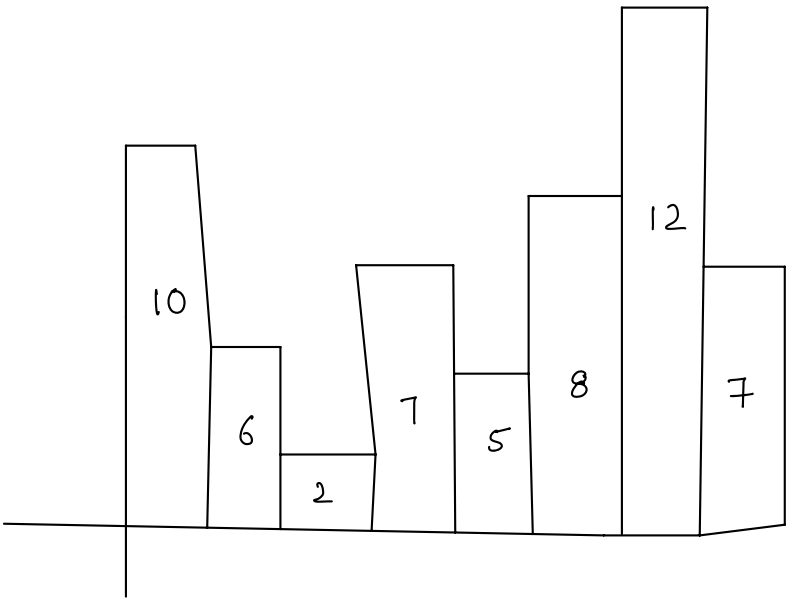
Water stored over building 2 ?

max on right = 12

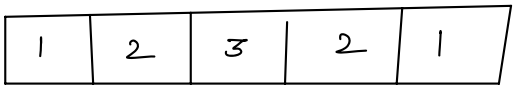
" " left = 10

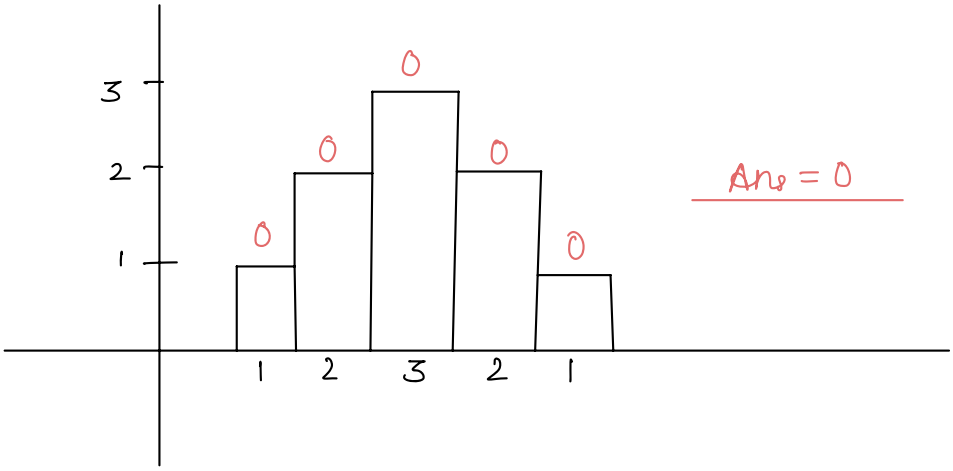
min (10, 12) − 2

= 10 − 2 = 8.



Water stored over building 5?





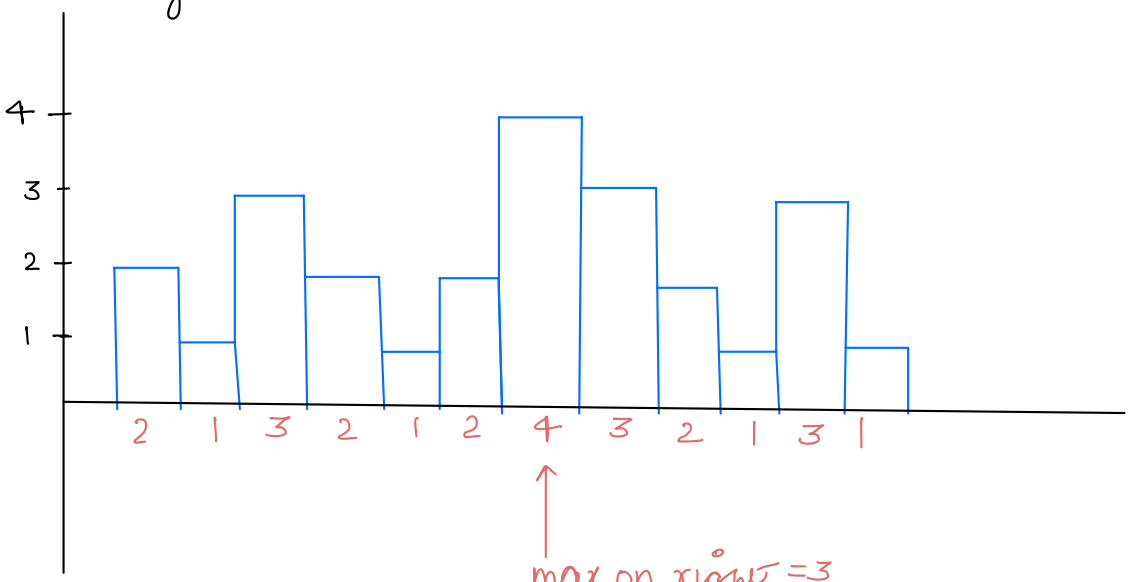
Ans = 0

Brute force

```
ans = 0
for( i = 1 ——— n-2) {
        maxL = findMaxLeft ( A , 0 , i-1 ); —— O(n)
        maxR = findMaxRight(A , i+1 , n-1) —— O(n)
        water = min ( maxL , maxR) — A[i].
        if ( water > 0) {
            ans += water;
        }
}
return ans;
```

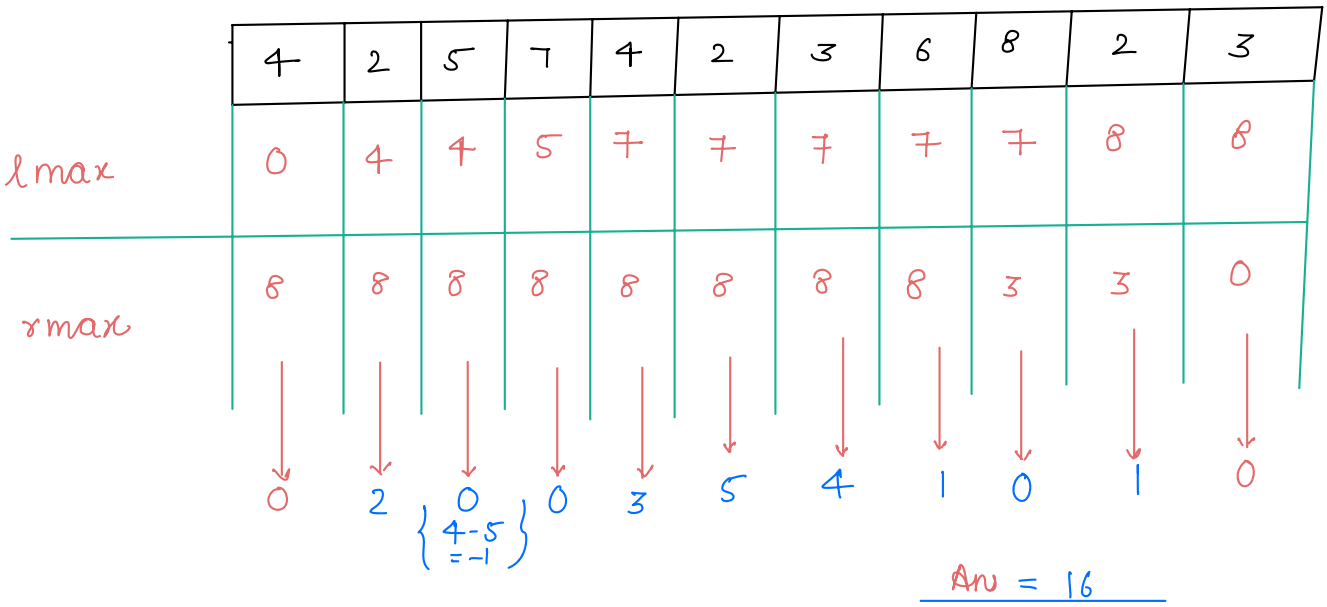
TC: $O(n^2)$
SC: $O(1)$

Edge case:



max on right = 3
" " left = 3
ans = min (3, 3) − 4
= 3 − 4 = −1

Optimised Approach



Ans = 16

$i = 0$ $lmax[i] = max[lmax[i-1], A[i-1]]$

$i = n-1$ $rmax[i] = max[rmax[i+1], A[i+1]]$

Pseudocode

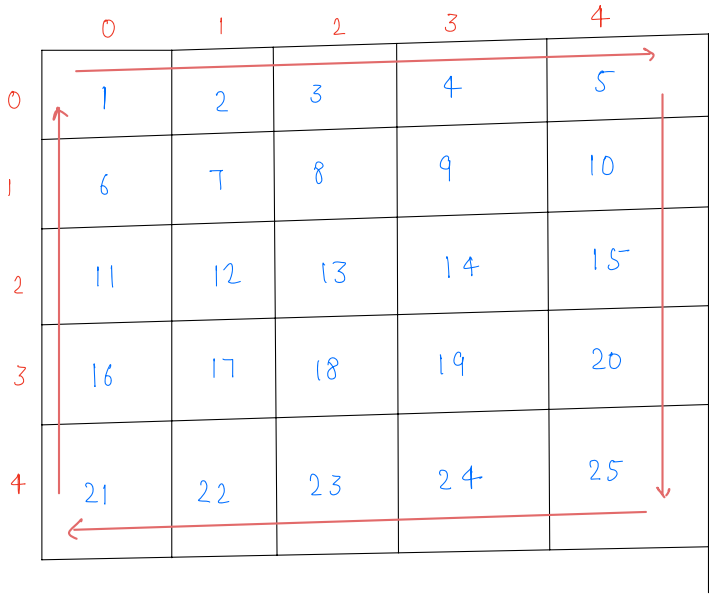
```
ans = 0
lmax[n] = { }
lmax[0] = 0
for( i = 1 ———— n-1) {
      lmax[i] = max( lmax[i-1], A[i-1]);
}
rmax[n] = { }
rmax[n-1] = 0
for( i = n-2 ———— 0) {
      rmax[i] = max[rmax[i+1], A[i+1]]
}
for( i = 1 ———— n-2) {
      water = min[ lmax[i], rmax[i] ] - A[i]
      if( water > 0) {
            ans += water;
      }
}
return ans
```

TC: O(n)

SC: O(n)

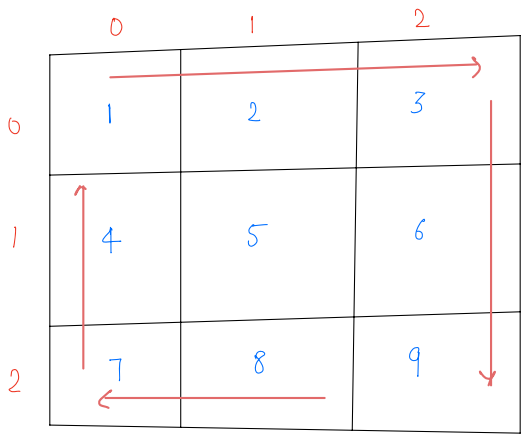
Problem 3 Boundary level printing

Given mat[n][n], print boundary elements in clockwise direction.



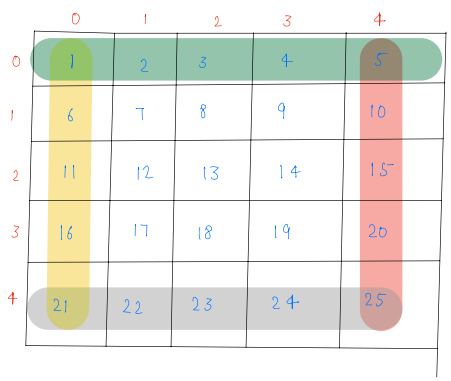
output

1 2 3 4 5
10 15 20 25
24 23 22 21
16 11 6



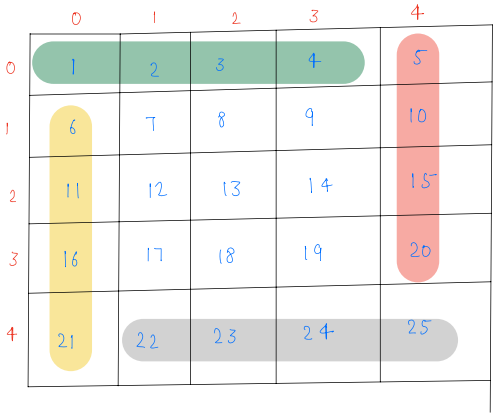
output

1 2 3
6 9
8 7
4



Approach

mat[5][5]



Green, Red, Silver, Yellow →

Green

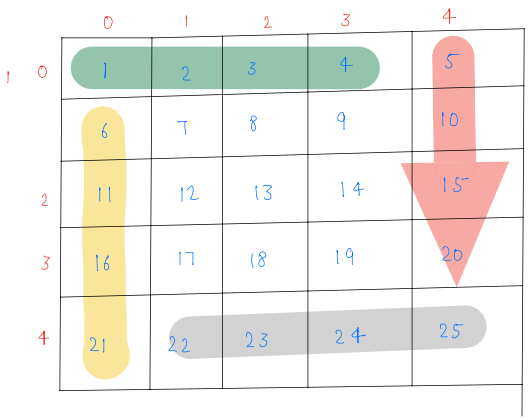
i	j	
i=0	j=0	1
i=0	j=1	2
i=0	j=2	3
i=0	j=3	4
i=0	j=4	

Red

i	j	
i=0	j=4	5
i=1	j=4	10
i=2	j=4	15
i=3	j=4	20
i=4	j=4	

Silver

i	j	
i=4	j=4	25
i=4	j=3	24
i=4	j=2	23
i=4	j=1	22
i=4	j=0	



Yellow

i	j	
i=4	j=0	21
i=3	j=0	16
i=2	j=0	11
i=1	j=0	6
i=0	j=0	

Algorithmic code

n = mat.length

```
void boundaryPrint(int[][] mat) {
```

Green

```
i=0, j=0
for(k=0; k<n-1; k++) {
    print(mat[i][j]);
    j++;
}
```

O(n)

// i=0 j=4

Red

```
for(k=0; k<n-1; k++) {
    print(mat[i][j]);
    i++;
}
```

O(n)

// i=4 j=4

Silver

```
for(k=0; k<n-1; k++) {
    print(mat[i][j]);
    j--;
}
```

O(n)

// i=4 . j=0

Yellow

```
for(k=0; k<n-1; k++) {
    print(mat[i][j]);
    i--;
}
```

O(n)

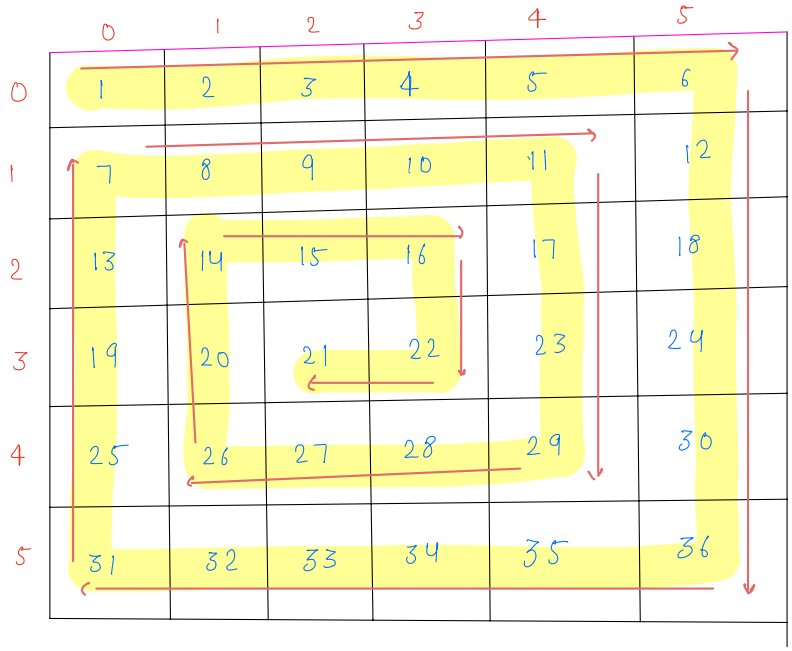
// i=0. j=0

```
}
```

TC: O(4*n) ≅ O(n)
SC: O(1)

Spiral order matrix

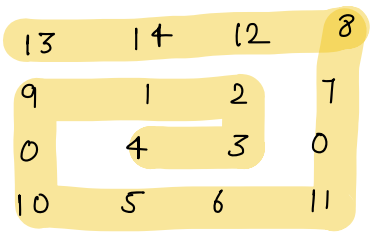
Given mat[n][n], print spiral order of matrix in clockwise direction.



output

1 2 3 4 5 6
12 18 24 30 36
35 34 33 32 31
25 19 13 7
8 9 10 11
17 23 29 28
27 26 20 14
15 16 22 21

Any

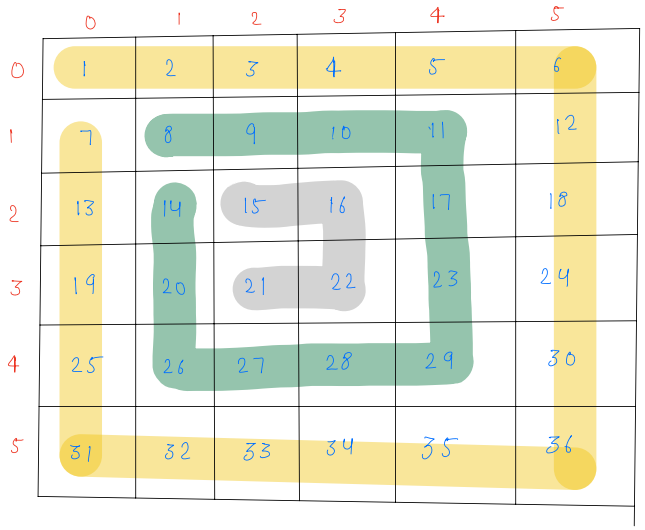


13 14 12 8
7 0 11 6
5 10 0 9
1 2 3 4

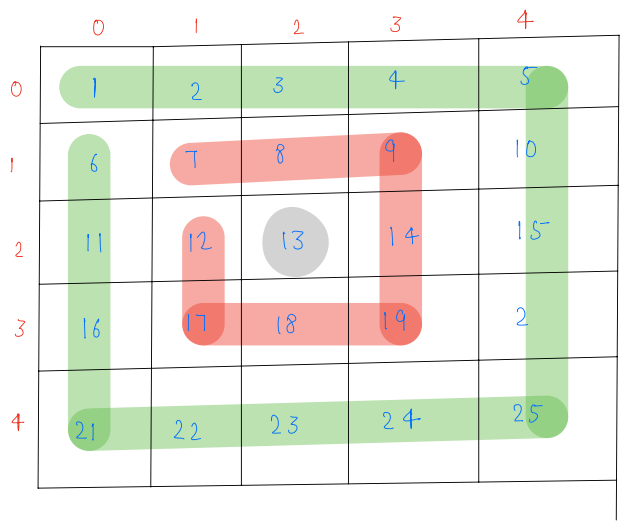
Any

Approach

mat[5][6]



	i	j	n Loop (k) = 5(n-1)
Yellow	0	0	k (5)
	i++	j++	k-=2
Green	1	1	3
	i++	j++	k-=2
Silver	2	2	1



	i	j	Loop (k)
Green	0	0	4 (n-1)
	i++	j++	k-=2
Red	1	1	2
	i++	j++	k-=2
Silver	2	2	0

Algorithmic code

```
void spiralorder(int[][] mat) {
    n = mat.length
    r = 0 , c = 0
  loop = n-1;
    while (loop > 0) {

        i = r . j = c.
        for(k=0 ; k<loop; k++) {
            print(mat[i][j]);
            j++;
        }
        for(k=0 ; k<~~n-1~~ loop; k++) {
            print(mat[i][j]);
            i++;
        }
        for(k=0 ; k<~~n-1~~ loop; k++) {
            print(mat[i][j]);
            j--;
        }
        for(k=0 ; k<~~n-1~~ loop; k++) {
            print(mat[i][j]);
            i--;
        }
        r++;
        c++;
        loop = loop-2;
    }
    if(loop==0) {
        print(mat[r][c];
    }
}
```

TC: $O(n^2)$
SC: $O(1)$

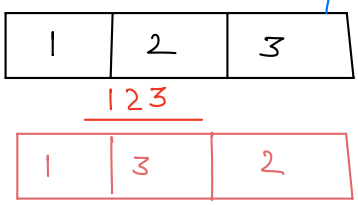
Break: 10:27 – 10:37

Qu Implement next permutation which rearranges numbers into numerically greater permutation of numbers for a given array of size n.

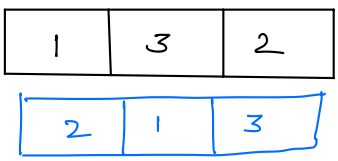
If such arrangement is not possible, it must be rearranged as lowest possible order. [0 - 9]

Expectation TC: O(n)
SC: O(1)

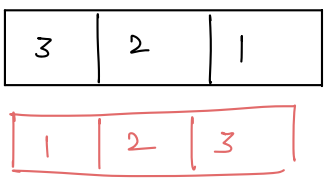
Example



1 2 3 — 123
1 3 2 132 ✓
2 1 3 213
2 3 1 231
3 1 2 312
3 2 1 321



1 3 2
1 2 3
2 1 3 ✓
2 3 1
3 1 2
3 2 1

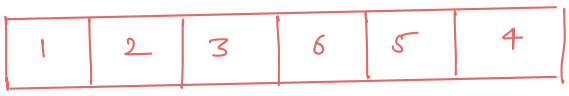


1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1

Idea

1. Think of permutation as a number

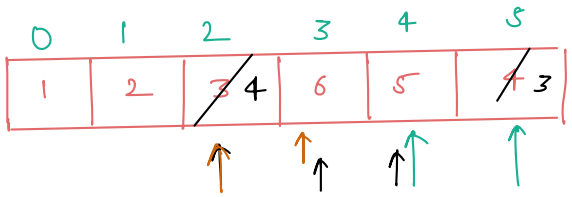
2.



——— Right to left

Ans ⇒ 1 2 4 3 5 6

3.



Traverse from right to left

```
i = n-2
while (i >= 0) {
    if (A[i] < A[i+1]) {
        break;
    }
}
```

i = 2

i = 3 ——— n-1.

find the smallest el greater than 3rd idx

min = ∞ , minIdx ———

```
for(j = i+1 ——— n-1) {
    if(A[j] > A[i] &&
       A[j] < min) {

    }
```

i = 2 j = 5

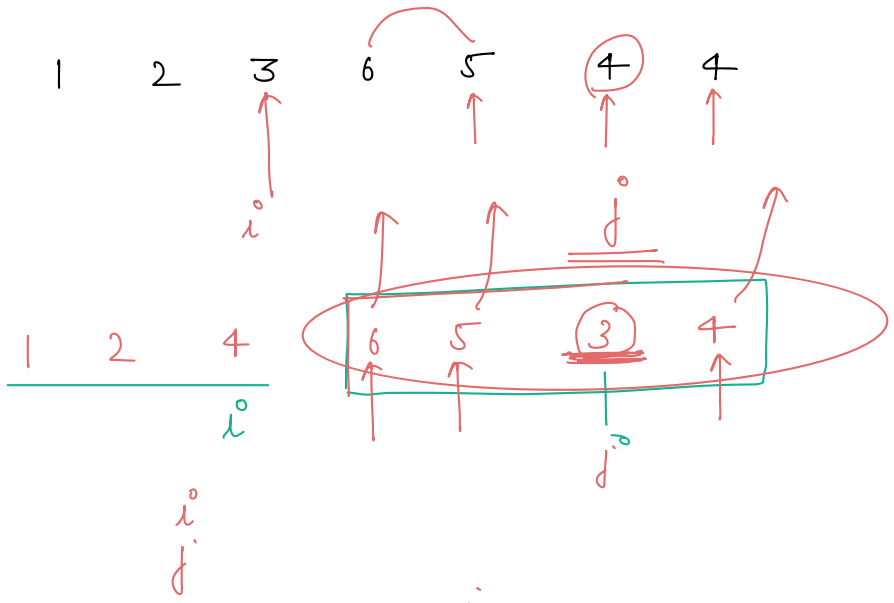
swap(A, i, j);

Reverse array after pivot to get smallest permutation

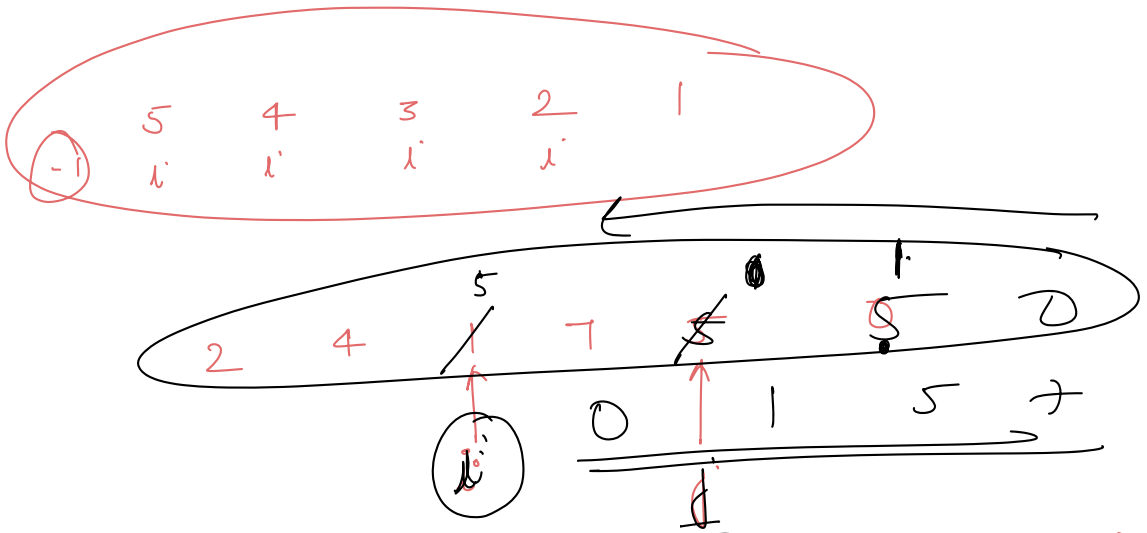
sorted order left desc

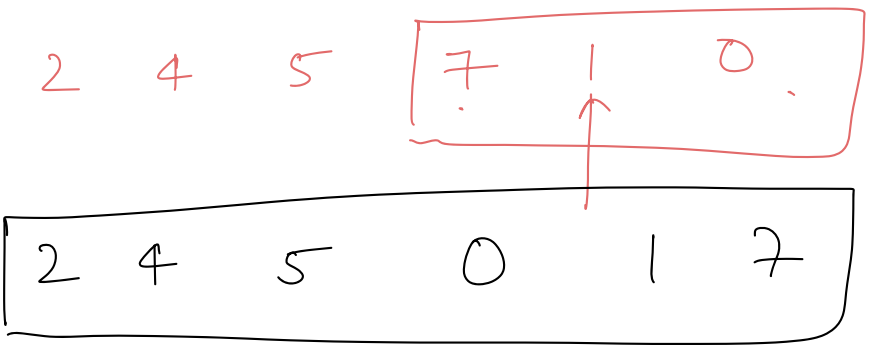
Thankyou

Doubts



$i = n$





2 4 5 7 1 1 0